

DOCUMENTACIÓN Y PRUEBAS DE LA API ALQUILAYA

Universidad Cristóbal Colón
Estudiante: Erick Gabriel Hernández Rebolledo
Docente: Jesús Leonardo López Hernández
Materia: Arquitectura Orientada A Servicios
Fecha: 14-10-2026

1. Enlace al repositorio

El código completo del proyecto se encuentra disponible en el siguiente repositorio de GitHub:

https://github.com/ErickGHR/alquilaya-api

En el repositorio se encuentra el código fuente, la configuración del proyecto, las pruebas automáticas, las instrucciones de ejecución y la colección utilizada en Postman.

2. Descripción de la API

AlquilaYa es una API REST desarrollada para administrar el funcionamiento básico de una empresa dedicada al alquiler de vehículos.

La aplicación permite registrar clientes, vehículos, oficinas, alquileres y mantenimientos. También relaciona cada vehículo con una oficina y cada alquiler con el cliente y el vehículo correspondiente.

Una de sus funciones principales es calcular automáticamente el precio total del alquiler tomando en cuenta la tarifa diaria y la cantidad de días seleccionados. La aplicación también evita que se registre un alquiler cuando el vehículo no está disponible o cuando las fechas coinciden con otro alquiler.

El proyecto está organizado en modelos, repositorios, servicios y controladores. Esta separación ayuda a mantener el código ordenado y facilita su modificación.

3. Tecnologías utilizadas

Para desarrollar y probar el proyecto utilicé las siguientes herramientas:

- Java 21.
- Spring Boot.
- Spring Web.
- Spring Data JPA.
- MySQL.
- Maven.
- Postman.
- Git y GitHub.

4. Funciones principales

La API cuenta con los siguientes recursos:

- Oficinas de alquiler.
- Vehículos.
- Clientes.
- Alquileres.
- Mantenimientos.

Las rutas principales son las siguientes:

/api/rental-offices

/api/vehicles

/api/customers

/api/rentals

/api/maintenances

Con estas rutas podemos hacer el registro, consulta, modificaciones y eliminación de datos.

5. Instrucciones para ejecutar el proyecto

Para ejecutar el proyecto de forma local se necesita tener instalado Java 21, Git y MySQL.

Primero se descarga el proyecto desde GitHub utilizando los siguientes comandos:

git clone https://github.com/ErickGHR/alquilaya-api.git

cd alquilaya-api

Después se inicia MySQL y se ejecuta el script incluido en el proyecto:

.\run-local.ps1

El script solicita el usuario y la contraseña de MySQL. Después de establecer la conexión, la aplicación crea automáticamente la base de datos alquilaya.

Cuando la aplicación termina de iniciar, la API queda disponible en la siguiente dirección:

http://localhost:8080

Para detenerla se utiliza la combinación **Ctrl + C** en la terminal.

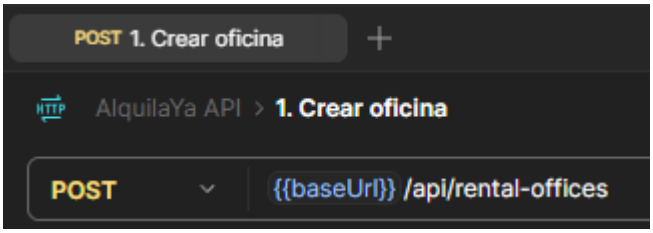
6. Pruebas realizadas en Postman

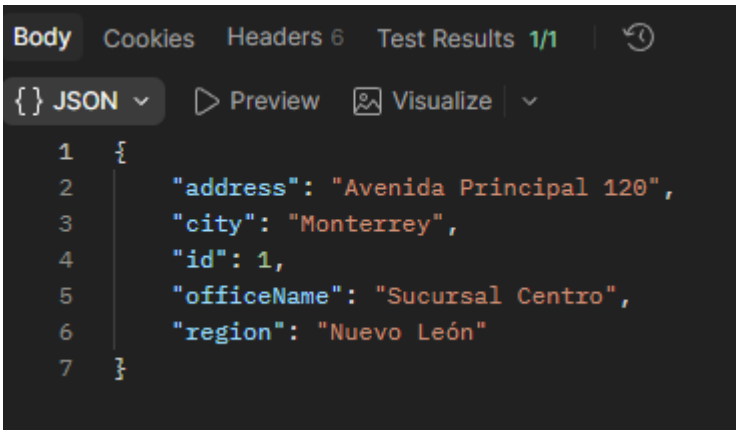
Para comprobar el funcionamiento de la API utilicé una colección de Postman con seis peticiones. Las pruebas se realizaron en orden porque algunos registros dependen de los identificadores generados anteriormente.

6.1 Registro de una oficina

Primero registré una oficina de alquiler. En la petición envié el nombre de la sucursal, la dirección, la ciudad y la región.

La API respondió con el código 201 Created y asignó automáticamente un identificador a la oficina.

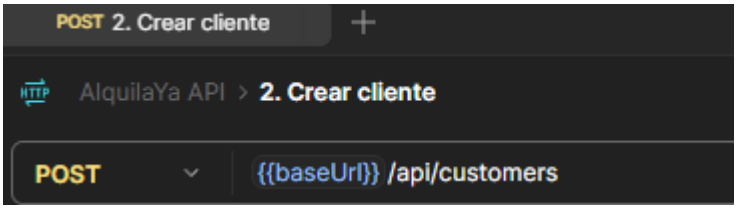


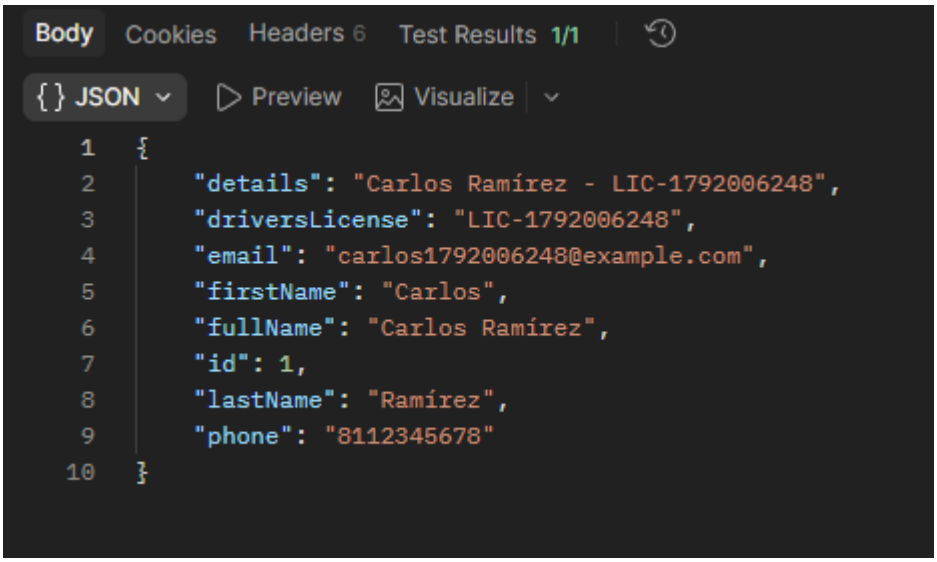


6.2 Registro de un cliente

En la segunda prueba registré un cliente con su nombre, apellidos, correo electrónico, licencia de conducir y teléfono.

La respuesta fue 201 Created, lo que indica que el cliente se guardó correctamente. La aplicación también generó su identificador.

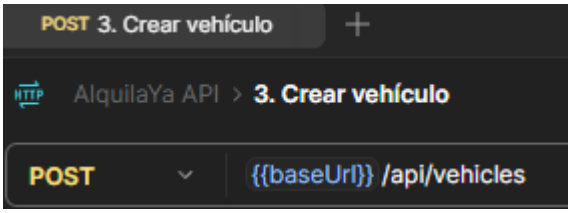


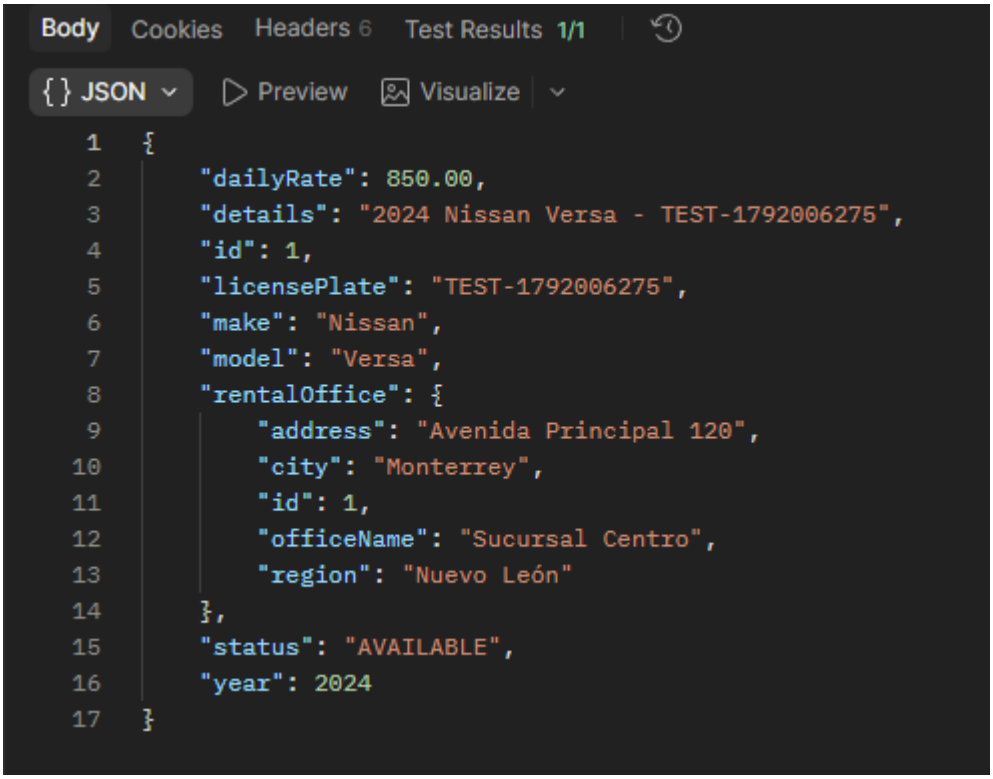


6.3 Registro de un vehículo

Después registré un vehículo y lo relacioné con la oficina creada en la primera prueba. Se enviaron datos como la marca, el modelo, el año, la placa, la tarifa diaria y el estado del vehículo.

La API respondió con 201 Created y almacenó el vehículo con el estado AVAILABLE, que indica que se encuentra disponible.

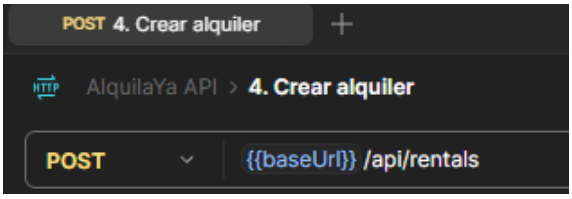


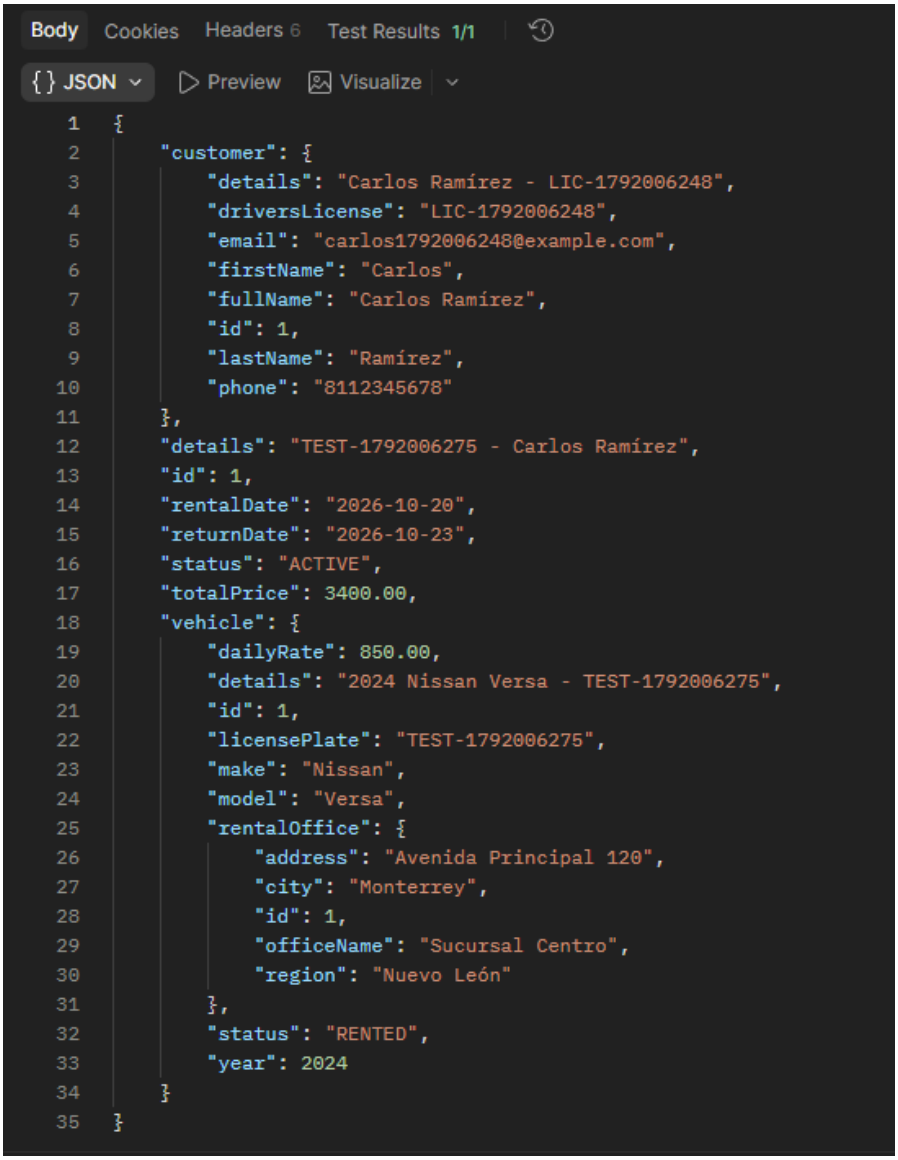


6.4 Creación de un alquiler

En esta prueba relacioné el vehículo con el cliente registrado anteriormente. También envié la fecha de inicio, la fecha de devolución y el estado del alquiler.

La aplicación calculó el precio total de forma automática utilizando la tarifa diaria del vehículo y la cantidad de días del alquiler. La respuesta obtenida fue 201 Created.

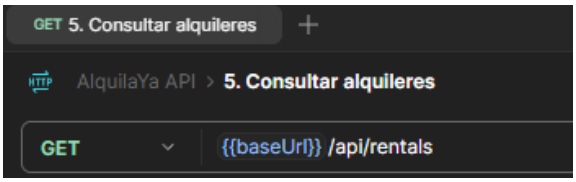




6.5 Consulta de alquileres

Después consulté la lista de alquileres registrados utilizando una petición GET.

La API respondió con el código 200 OK y mostró la información del alquiler, el cliente, el vehículo, las fechas y el precio total calculado.

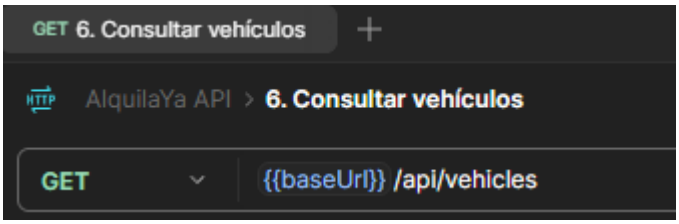


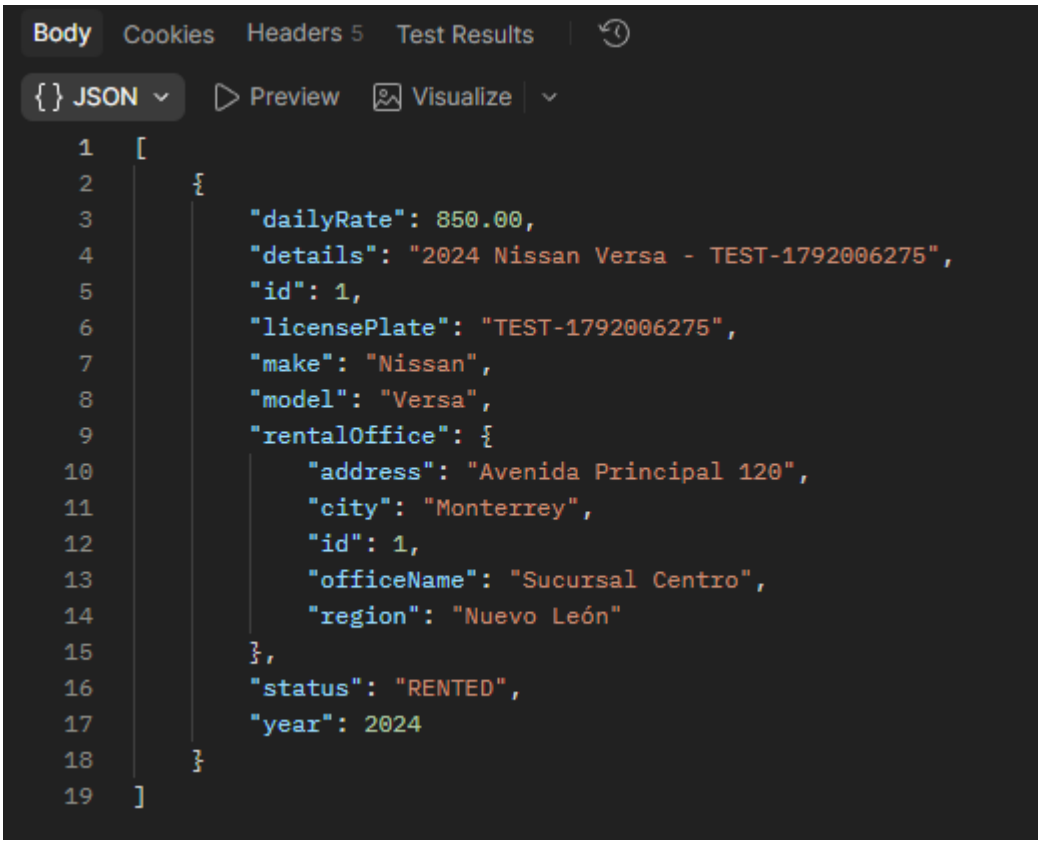


6.6 Consulta de vehículos

Finalmente consulté la lista de vehículos.

La respuesta fue 200 OK y mostró los datos del vehículo registrado, incluyendo su oficina, tarifa diaria y estado actual.





7. Resultado de las pruebas

Las seis peticiones realizadas en Postman funcionaron correctamente. Los registros se guardaron en la base de datos y las consultas devolvieron la información esperada.

También se comprobó que las relaciones entre oficinas, vehículos, clientes y alquileres funcionan correctamente. El cálculo del precio total se realizó desde el servicio de la aplicación, por lo que no fue necesario enviarlo manualmente.

8. Conclusión

Con este proyecto pude realizar una API REST conectada a una base de datos MySQL. La aplicación permite administrar los datos principales de un sistema de alquiler de vehículos y aplicar reglas para evitar registros incorrectos.

Las pruebas de Postman ayudaron a comprobar que los endpoints responden correctamente y que la información se almacena y consulta de acuerdo con lo esperado.